Locally Checkable Problems in Distributed Computing

Dennis Olivetti

Last Update: June 18, 2026

Contents

Chapter 1

Introduction

The goal of this book is to provide an overview of recent results about locally checkable problems in the distributed setting. Locally checkable problems are problem for which, if we are given a solution, we can efficiently check its validity. We will mainly consider the LOCAL model of distributed computing, a synchronous message-passing model, where a graph represents a communication network, and neither the size of the messages nor the computational power of the machines is restricted. The considered complexity measure is the number of communication rounds required to solve a given problem. We will address the following topics:

- Distributed Complexity Theory.
  - What are the possible distributed time complexities of locally checkable problems?
  - Does randomness help? How much? Is shared randomness more powerful than private randomness? How about quantum?
  - Can we automate our work, that is, can we automatically determine the complexity of a given problem?
- Lower Bound Techniques.
  - We will mainly talk about the round elimination technique, which has been used to prove many lower bounds over the years.
  - We will discuss other techniques, such as Mark's technique, KMW, and indistinguishability arguments.
- Connections with other fields.
  - There are deep connections between the distributed setting and a mathematical field called *descriptive combinatorics*.
  - There are deep connetions bewtween the distributed setting and *finitary factors of iid processes*, a topic coming from the area of analysis of randomized processes.

Chapter 2

Preliminaries

2.1 The LOCAL model

We model a network as a graph $G = (V, E)$, where the nodes represent machines, and the edges represent communication links. Nodes may be assigned unique IDs. This model works as follows:

- Time proceeds in synchronous steps (machines share a clock).
- At the beginning, each node only knows its own input and its degree. The input of a node can be, e.g., its ID, the knowledge of the size $n = |V|$ of the graph, the maximum degree Δ, and/or some additional problem-specific inputs.
- Then, computation proceeds in rounds. At each round, each node can exchange messages with its neighbors, and update its own local state (each node has its own private memory, no memory is shared between nodes).
- Each node must eventually produce an output.

The complexity of an algorithm is the worst-case number of rounds that it requires to terminate. This complexity is often measured as a function of n and Δ.

In the LOCAL model, messages can be arbitrarily large, and computation is unbounded. There are many variants of the LOCAL model that one can consider, and they may have different power:

- Is n known by the nodes? Is a polynomial upper bound on n known?
- Do nodes have IDs?
- Do nodes have access to randomness? How much randomness? A finite or infinite bit-string? Do we require Las-Vegas or Monte-Carlo algorithms?
- Messages are arbitrarily large, but are they finite? Can we e.g. send real numbers?
- Can nodes locally perform uncomputable computation?

2.2 Locally Checkable Problems

Locally Checkable Problems (LCP) are problems for which it is easy to check whether a given solution is valid. In more detail, a locally checkable problem is a problem for which there exists a constant-time distributed algorithm called *verifier* satisfying that, given a solution:

- If the solution is correct, then the algorithm outputs *yes* on all nodes.
- If the solution is incorrect, then the algorithm outputs *no* on at least one node.

Many interesting problems are locally checkable, for example:

- Coloring problems. For example, consider the $(\Delta + 1)$-coloring problem, and assume that the verifier knows Δ. The verifier requires 1 round of communication, in which it checks that the color of the current node is in $\{1, \ldots, \Delta + 1\}$, and that it is different from the color of the neighbors.
- Maximal independent set. The verifier needs to check that if the current node is in, then all its neighbors must be out, and that if the current node is out, then at least one neighbor is in. Observe that the *Maximum* independent set problem is not locally checkable.
- Single source shortest paths (SSSP), or at least some variants of it. Consider for example the variant in which each node outputs its own distance and a pointer to the parent. Then the verifier needs to check that the distance of the parent, plus the weight of the edge connecting the node to the parent, is equal to the distance of the node, and that no other neighbor would give a strictly smaller distance.
- Sinkless orientation, that is, orienting the edges so that each node does not have all edges oriented incoming.
- List-coloring, where each node is given a list of colors as input, and each node needs to output a color from its own list, such that the resulting coloring is proper.

2.3 LCLs

In [12], a restricted class of LCP, called Locally Checkable Labelings (LCLs), was introduced. This class is restricted enough so that we can prove interesting results, but wide enough to still contain many problems of interest. In more detail, LCLs are LCPs with the following restrictions:

- The number of input and output labels is constant (i.e., it does not depend on n).
- It is assumed that $\Delta \in O(1)$.

An example of a problem that is an LCP but not an LCL is SSSP, because nodes need to output distances, and distances could depend on n.

The nice thing about this class of problems is that each problem has a finite description:

- Let r be the number of rounds taken by the verifier. Clearly, the output of the verifier can only depend on things at distance at most r from the node.
- Since Δ and r are in $O(1)$, and the number of labels is constant, there is a finite number of graphs that the verifier could see.
- Thus, we can list the ones that the verifier would accept. Hence, we can describe a problem by listing the accepted neighborhoods.

2.4 Distributed Complexity Theory

Being LCLs so restrictive, researchers managed to prove many interesting properties about them. Moreover, many techniques that have been initially developed to understand specific LCLs, have then been generalized to understand problems that fall outside of this class (e.g., round elimination). Common questions are the following:

- What are possible LCL complexities?
- Can we automatically decide the complexity of a given LCL?

- Does shared/private randomness help?
- Does quantum help?

2.4.1 Paths and Cycles

In the case of paths and cycles, LCLs have 3 possible complexities [1, 7, 11]:

- $O(1)$. Problems with this complexity often admit very easy algorithms. In the case of directed cycles with no inputs, we even know that all problems with this complexity can be solved in 0 rounds.
- $\Theta(\log^\star n)$. Problems with this complexity require breaking symmetry, but do not require global coordination. Examples are 3-coloring and MIS.
- $\Theta(n)$ and unsolvable problems. Problems with this complexity require global coordination. An example is 2-coloring, where, by fixing the output of a single node, we are forcing the output on all other nodes.

Surprisingly, LCLs on paths and cycles cannot have any other possible complexity, and randomization never helps. This fact has the following interesting implication: you can be sloppy. Suppose you prove that your favourite problem can be solved in $O(\log n)$ rounds via a simple randomized algorithm. You automatically get that the problem can also be solved in $O(\log^\star n)$ by a more clever algorithm, deterministically.

Another surprising fact is that everything is decidable, even for LCLs with inputs. In other words, we can feed the description of an LCL to a computer program, and the program is going to output the correct complexity, and a description of an optimal algorithm.

2.4.2 Trees

In the case of trees, there are the same possible complexities of paths and cycles, plus some more [10, 2, 8]:

- $\Theta(\log n)$ deterministic. These problems can either be $\Theta(\log n)$ randomized, or $\Theta(\log \log n)$ randomized.
- $\Theta(n^{1/k})$ for any $k \in \mathbb{N}^+$. For these problems, randomness does not help.

No other complexity is possible. Moreover, it is decidable whether a problem can be solved in $O(\log n)$ or not, and in the latter case, what the exact value of k is. Summarizing, some things are still decidable, and randomness can only help exponentially or not at all, and if it helps, it only helps for problems with deterministic complexity $\Theta(\log n)$. A major open question is whether it is decidable if a problem is $\Omega(\log n)$.

2.4.3 General graphs

In general graphs, things are entirely different. For deterministic complexities, the following is known [9, 5, 2]:

- There are problems with complexity $O(1)$.
- No problem with complexity in $\omega(1) - o(\log \log^\star n)$ exists.
- The region $\Omega(\log \log^\star n) - O(\log^\star n)$ is dense: informally, for any complexity in this range, we can find a problem with a complexity that is arbitrarily near to it.
- No problem with complexity in $\omega(\log^\star n) - o(\log n)$ exists.

- The region $\Omega(\log n)$ is dense.

In the case of general graphs, undecidability results are known. Very informally, they are proved by showing that it is possible to define problems that require outputting the execution of a Turing machine, and whether the problem is constant or not depends on the running time of the machine. About randomness, a surprising connection is known: if a problem has randomized complexity $o(\log n)$, then it has randomized complexity $O(T_{\text{LLL}})$, where T_{LLL} is the distributed time complexity of the constructive Lovász Local Lemma [10]. It is known that T_{LLL} is in $\Omega(\log \log n)$ and in $O(\text{poly} \log \log n)$, and a big open question is determining the exact complexity of LLL. Moreover, it is known that randomness can help in different regions. For example, it is known that, for all $k \in \mathbb{N}$, there are problems with deterministic complexity $\Theta(\log^{k+1} n)$ and randomized complexity $\Theta(\log^k n \log \log n)$ [3].

2.5 Easy vs Hard

A central question about LCLs is to determine whether a problem is *easy* or *hard*, which we define as follows.

- Easy problems are problems that can be solved in $O(\log^\star n)$. We call these problems easy because, in order to solve them, we do not need to exploit structural properties of the graph, and only some basic symmetry breaking is required. In some sense, these problems are truly local.

- If a problem cannot be solved in $O(\log^\star n)$, then we know it must have deterministic complexity $\Omega(\log n)$. If a problem falls in this latter case, then we say that the problem is hard. Problems of this type cannot be solved with local information: an algorithm running in $\Omega(\log n)$ rounds (for some large-enough constant hidden in the Ω notation) is guaranteed to see at least a node of degree ≤ 2 or a cycle. Algorithms of this type are not truly local; they need to see *something* in order to be able to solve the problem. Moreover, problems of this type have randomized complexity $\Omega(\log \log n)$. Observe that this is the sweet spot to obtain high-probability guarantees: either the algorithm sees *something* in the structure of the graph that it may be able to exploit, or it sees $\Omega(\log n)$ nodes. In the latter case, consider some randomized process that has constant success probability independently on each node. By seeing $\Omega(\log n)$ nodes, the algorithm will see a successful event with high probability, which it may be able to exploit.

2.6 Black-White formalism

An important technique used to prove lower bounds in the LOCAL model is the Round Elimination technique [6]. This technique does not directly work on LCLs, but requires the problem to be described with a specific language, called black-white formalism. It turns out that, at least on trees, this is not a restriction: we can convert any given LCL into a problem in the black-white formalism, and the two problems are equivalent, up to $O(1)$ additive rounds [4]. On general graphs, problems in the black-white formalism are a strict subclass of LCLs, but most natural problems can anyway be expressed in the black-white formalism.

Chapter 3

Distributed Complexity Theory

[TODO: summarize all papers about LCLs, with proof sketches]

Chapter 4

Connections with Other Fields

[TODO: descriptive combinatorics, ffiid]

Chapter 5

Black-White Formalism

The black-white formalism allows to define (some) LCLs in a concise way. While there are many variants that we will consider later, the most basic setting is the following:

- We assume that we are in a (Δ, δ)-biregular graph, where white nodes have degree Δ and black nodes have degree δ.
- There are no input labels.
- We need to output one label on each edge.
- We have a set (called *white constraint*) of *white allowed configurations*, which are multisets of size Δ that denote valid labelings of the edges incident to the white nodes.
- We have a set (*black constraint*) of *black allowed configurations*, which are multisets of size δ that denote valid labelings of the edges incident to the black nodes.

In other words, a problem Π in the black-white formalism is a triple (Σ, W, B), where Σ is the (finite) set of possible labels, W is a finite multiset, where each multiset has size Δ, and B is a multiset, where each multiset has size δ. Note that the definition of a problem Π is just this, in particular Σ is finite and there is no n involved: we cannot define problems where the number of labels depends the size of the graph.

5.1 Sinkless Orientation on Bipartite Graphs

Let's see a simple example (illustrated in Figure 5.1). Consider the following setting: we have the promise that our graph is 3-regular and 2-colored. We want to define a problem that requires to orient the edges of the graph such that no node is a sink, that is, each node must have at least one edge oriented outwards. We can define this problem in the black-white formalism as follows.

- $\Sigma = \{\mathsf{A}, \mathsf{B}\}$
- $W = \{\{\mathsf{A}, \mathsf{A}, \mathsf{A}\}, \{\mathsf{A}, \mathsf{A}, \mathsf{B}\}, \{\mathsf{A}, \mathsf{B}, \mathsf{B}\}\}$
- $B = \{\{\mathsf{B}, \mathsf{B}, \mathsf{B}\}, \{\mathsf{B}, \mathsf{B}, \mathsf{A}\}, \{\mathsf{B}, \mathsf{A}, \mathsf{A}\}\}$

The intuition is the following:

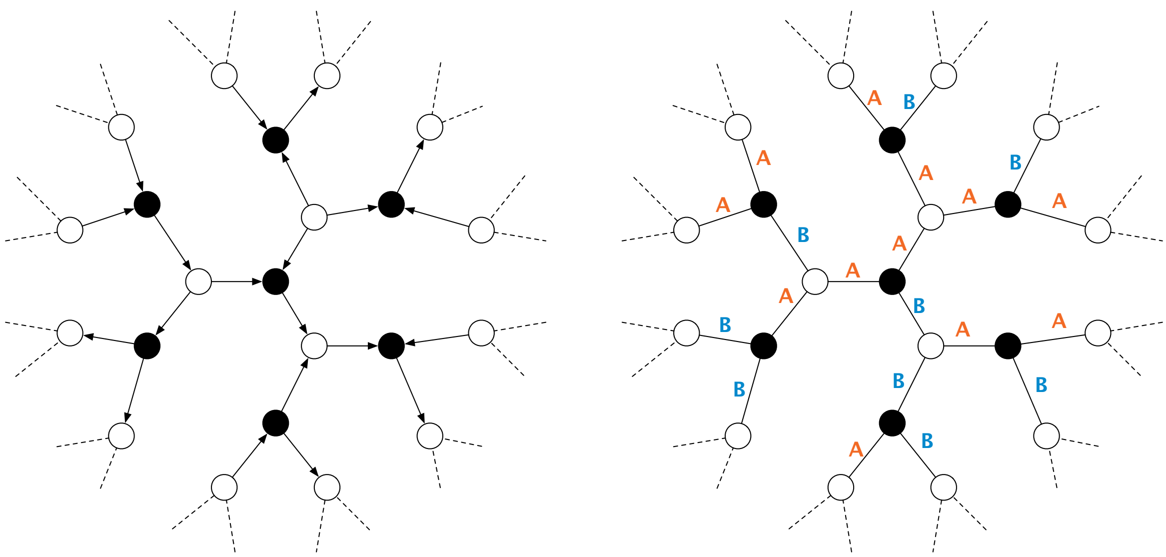
- Each edge gets exactly one label, A or B. If an edge is labeled A, then the edge is oriented from the white node to the black node, while if an edge is labeled B, then the edge is oriented from the black node to the white node.
- The white constraint W allows every configuration of 3 labels that contains at least one A, that is, at least one edge oriented away from the white node.

- The black constraint B allows every configuration of 3 labels that contains at least one B, that is, at least one edge oriented away from the black node.

In order to describe problems in the black-white formalism more compactly, we use the following notation. Instead of writing, e.g., $\{\mathsf{A}, \mathsf{A}, \mathsf{A}\}$, we simply write $\mathsf{A}\ \mathsf{A}\ \mathsf{A}$, or even A^3. Hence, the white constraint becomes $\{\{\mathsf{A}^3\}, \{\mathsf{A}^2\ \mathsf{B}\}, \{\mathsf{A}\ \mathsf{B}^2\}\}$. To make the notation even more compact, we write these three allowed configurations as a single one: $\mathsf{A}\ [\mathsf{AB}]^2$. Think of it as a regular expression: we pick an A, and then we can pick A or B, twice. We call this type of configurations, *condensed configurations*. Sometimes we even omit the square brackets, and we just write $\mathsf{A}\ \mathsf{AB}^2$. This is the format that *round eliminator* (the tool that automatically applies the round elimination technique) expects. In order to describe a whole problem compactly, we omit Σ, we write a condensed configuration on each line, and we separate the white and the black constraint by an empty line. Hence, sinkless orientation is compactly defined as follows:

$\mathsf{A}\ \mathsf{AB}^2$

$\mathsf{B}\ \mathsf{AB}^2$



(a) A solution of bipartite sinkless orientation on bipartite 2-colored graphs.

(b) The same solution, encoded in the black-white formalism.

Figure 5.1: On the left, a solution of bipartite sinkless orientation as one would naturally imagine it. On the right, the same solution encoded in the black-white formalism. Each edge oriented from a white node to a black node gets the label A, while each edge oriented from a black node to a white node gets the label B.

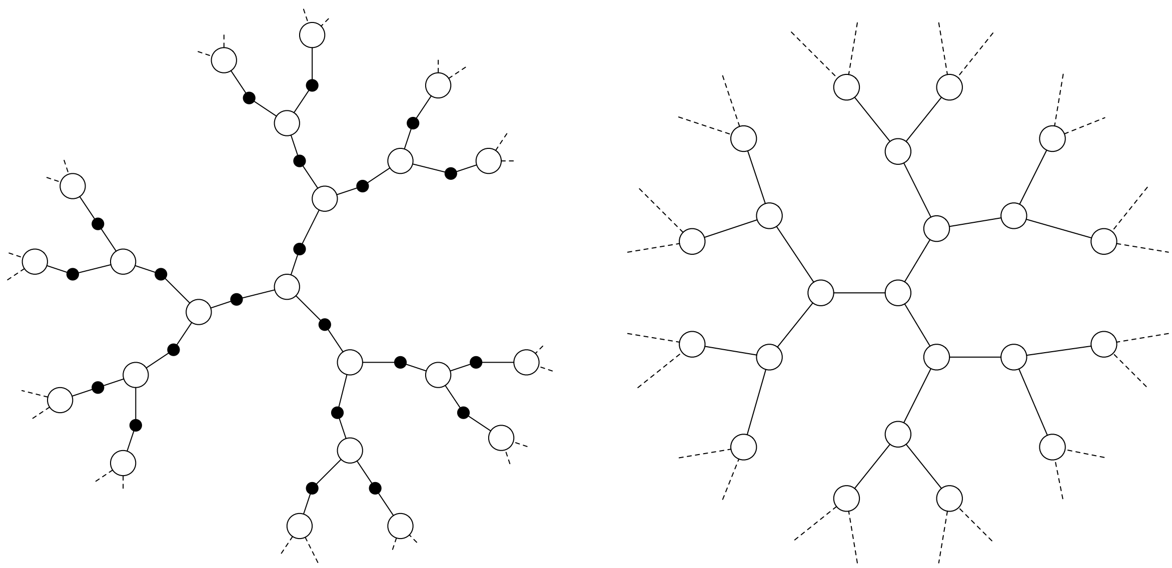
5.2 Node-Edge Formalism

The above example was in bipartite 2-colored graphs. However, the same formalism can be used to define problems in standard graphs. The idea is to just take $\delta = 2$ (see Figure 5.2). Observe that, in this case, each black node is in the middle of an edge connecting two white nodes. Let's just ignore these black nodes and pretend they do not exist. We get the following setting:

- We assume that we are in a Δ-regular graph.

- There are no input labels.
- We need to output one label on each *half-edge* (node-edge pair). In other words, we need to output two labels for each edge, one from each side.
- We have a list of *allowed node configurations*, which are multisets of size Δ that denote valid labelings of the edges incident to the nodes.
- We have a list of *allowed edge configurations*, which are multisets of size 2 that denote valid labelings of the edges.

This special case of the black-white formalism is called *node-edge formalism*.



(a) A $(3, 2)$-biregular graph.

(b) A 3-regular graph.

Figure 5.2: By considering the graph on the left, and ignoring black nodes, we obtain the graph on the right. Observe that a labeling of the edges of the graph on the left corresponds to a labeling of the half-edges of the graph on the right.

5.3 Sinkless Orientation

Let's see an example of a problem encoded in the node-edge formalism (see Figure 5.3). We can define sinkless orientation as follows:

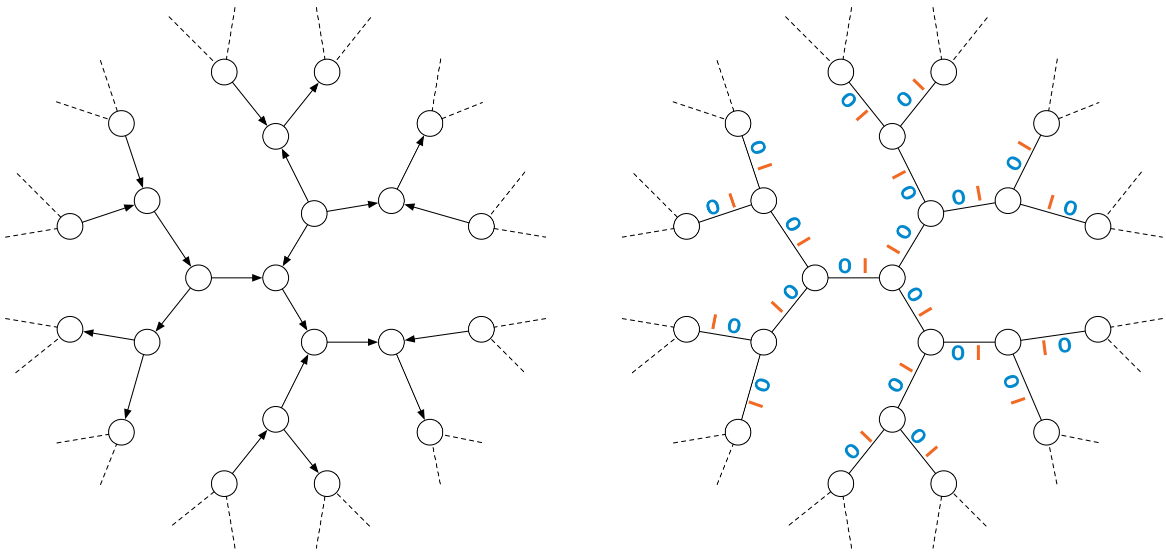
$\mathsf{O}\ [\mathsf{IO}]^2$

$\mathsf{I}\ \mathsf{O}$

In other words:

- The labels are I (oriented incoming) and O (oriented outgoing).
- Each node must have at least one O, that is, one edge oriented outgoing. But also two or three outgoing edges are fine, so all choices over the regular expression $\mathsf{O}\ [\mathsf{IO}]^2$, that is, $\mathsf{O}\ \mathsf{I}\ \mathsf{I}$, $\mathsf{O}\ \mathsf{O}\ \mathsf{I}$, $\mathsf{O}\ \mathsf{I}\ \mathsf{O}$, $\mathsf{O}\ \mathsf{O}\ \mathsf{O}$ are allowed. Note that $\mathsf{O}\ \mathsf{O}\ I$ and $\mathsf{O}\ \mathsf{I}\ \mathsf{O}$ are the same multiset, because the order does not matter.

- Edges must be properly oriented: if a node labels an edge I, then the other endpoint of the edge must label the edge O, and vice versa. We do not write O I in the list of allowed configurations because it is the same multiset as I O, and the order does not matter.



(a) A solution of sinkless orientation.

(b) The same solution, encoded in the node-edge formalism.

Figure 5.3: On the left, a solution of sinkless orientation as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Each edge gets the label O on the side oriented outgoing and O on the side oriented incoming.

5.4 Coloring

Let's see another example, 3-coloring 3-regular graphs (see Figure 5.4). We can define it in the node-edge formalism as follows.

R^3

G^3

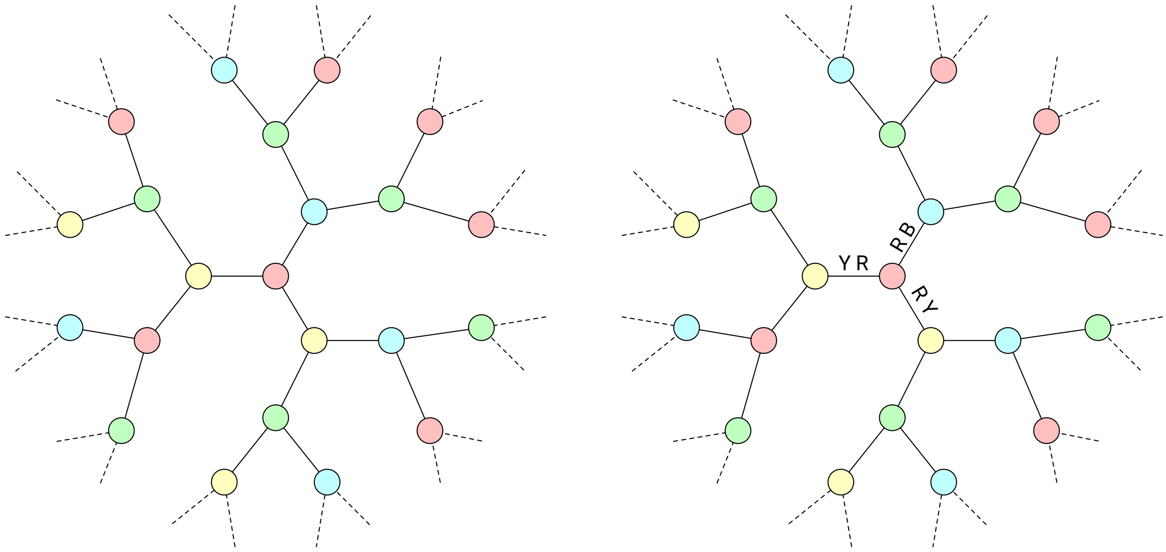
B^3

R GB

G B

In other words:

- Nodes can be colored red, green, or blue.
- Nodes colored, e.g., red, must output R on each of the three incident half-edges.
- On the edges, any configuration not consisting of the same color twice is allowed, that is, R G, R B, and G B are allowed, but, e.g., R R is not.



(a) A solution of 3-coloring in 3-regular graphs.

(b) An example of encoding in the node-edge formalism. The central node would, e.g., output R R R.

Figure 5.4: A solution of 3-coloring in 3-regular graphs, as one would naturally imagine it, and how it is encoded in the node-edge formalism.

5.5 Maximal Independent Set

Let's now consider a problem that is slightly harder to encode, Maximal Independent Set (MIS). In this problem, we need to select a subset of nodes that satisfy independence and maximality. While it may feel natural to try to encode it by using two labels (I and O, which stand for "in the set" and "out, not in the set"), this is not possible. In more detail, we could try to require nodes in the set to output I on each incident half-edge, and nodes not in the set to output O on each incident half-edge. Note that this would not work:

- There are valid solutions in which two nodes not in the set are neighbors, and hence we would need to allow O O on the edges.
- Hence, the node constraint contains the configuration O ... O, while the edge constraint contains the configuration O O.
- This implies that the obtained problem can be solved in 0 rounds of communication, by outputting O everywhere.
- The obtained problem is not *maximal independent set*, but just *independent set*!

In other words, we need one more label to ensure maximality. We can define MIS in the node-edge formalism, on 3-regular graphs, as follows.

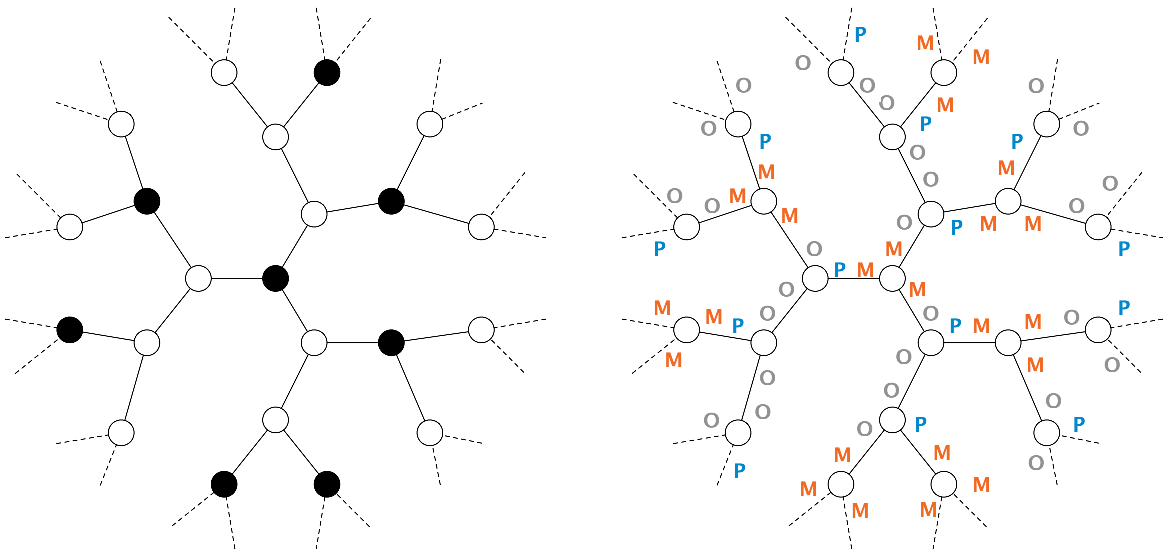
M^3
P O^2

M OP
O O

That is, MIS is encoded as follows:

- Nodes in the set output M^3.
- Nodes not in the set need to *prove* that they have at least one neighbor in the set, by outputting P towards them. On the other two edges, they output O.
- On the edges, we can see that P is only compatible with M (that is, the configuration M P is the only one containing P), this ensures maximality. Then, nodes not in the set could have more than one neighbor in the set, and hence we allow M O. Moreover, nodes not in the set could be neighbors, and hence we allow O O.

See Figure 5.5 for an example.



(a) A solution of MIS.

(b) The same solution, encoded in the node-edge formalism.

Figure 5.5: On the left, a solution of MIS in a 3-regular graph, as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Each node in the set outputs M^3, while each node not in the set outputs P towards a neighbor that is in the set, and O towards the other two neighbors.

Let's call the usual MIS problem standard-MIS, and the version encoded in the node-edge formalism encoded-MIS. It is important to notice that standard-MIS and encoded-MIS are not exactly the same problem:

- Given a solution to encoded-MIS, we can immediately infer a solution to standard-MIS.
- However, given a solution to standard-MIS, it takes 1 round of communication to obtain a solution to encoded-MIS, because nodes not in the set would not otherwise know where to output P.

We need to keep this in mind when proving lower bounds: if we prove that encoded-MIS requires at least T rounds to be solved, we only obtain a lower bound of $T - 1$ rounds for standard-MIS. While in the case of MIS it does not matter (asymptotically), for some other problems it could matter, so we have to keep this in mind.

5.6 Bipartite Maximal Matching

Let's see another example in the black-white formalism, by considering the problem of computing a maximal matching on bipartite 2-colored graphs (BMM). Similarly as in the case of MIS, one may be tempted to encode BMM by using two labels, but this is not possible, we need at least three. We can encode BMM, on $(3, 3)$-biregular graphs, as follows:

$\mathsf{M}\ \mathsf{O}^2$
P^3

$\mathsf{M}\ \mathsf{OP}^2$
O^3

See Figure 5.6 for an example. The intuition is the following:

- Edges that are part of the matching are labeled M.
- Then, if a white node is not matched, it needs to prove that all its neighbors are matched, so it points to all of them, by outputting P on all its incident edges.
- There are two types of black nodes, matched and unmatched. If a node is matched then it has exactly one edge labeled M, and then it can *accept* pointers. So all its other edges could be P or O. If a black node is not matched, then it needs to prove that all its white neighbors are matched. We do not need a new type of pointer for this, we can use O, because all non-matched edges incident to white matched nodes are labeled O.

Note that this is not the only possible way to encode BMM. One could use a more "symmetric" and intuitive version, as follows:

$\mathsf{M}\ \mathsf{PO}^2$
P^3

$\mathsf{M}\ \mathsf{PO}^2$
O^3

The intuition is the following:

- Matched edges get label M, then the other edges are either *white pointers* P or *black pointers* O.
- If a node is matched, then it outputs exactly one M, and then on the other edges everything else is allowed.
- If a white node is not matched, it outputs white pointers everywhere, that is, P^3.
- If a black node is not matched, it outputs black pointers everywhere, that is, O^3.
- Observe that if a white node is not matched, then all its edges are labeled P, and all black configurations containing P also contain M. Hence, all black neighbors of the white node are matched. The case of unmatched black nodes is symmetric.

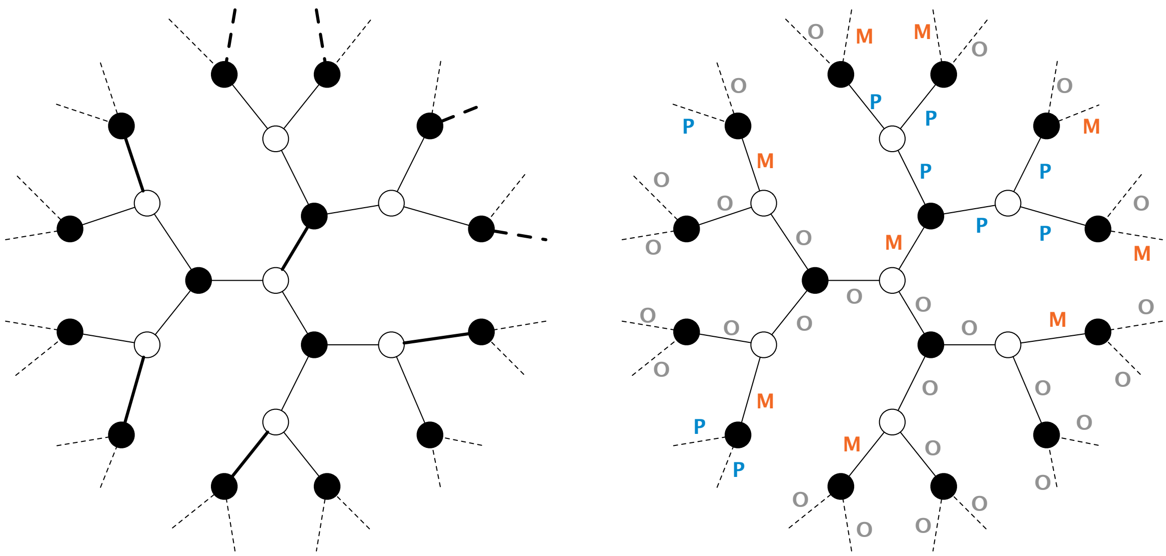
It turns out that the two variants that we have seen are equivalent, and it is now time to discuss an important detail: who outputs the labels? We will use the black-white formalism to prove upper and lower bounds via round elimination. In such a setting, it will be convenient to consider the following computational model:

- White nodes output labels on their incident edges.
- Black nodes participate in the computation, but they do not output anything.

- Hence, it is the job of the white nodes to output labels that satisfy, at the same time, white and black constraints on all nodes.

Now, a solution for the first variant *is also* a solution for the second variant. A solution for the second variant *can be converted*, without communication, into a solution for the first variant, if we consider the computational model just discussed: matched white nodes just need to change all their P into O. It is easy to see that the obtained solution is valid.

As a side note, if we enter the second variant of BMM into round eliminator, it automatically transforms the given problem into the first variant, because it sees the P in the condensed configuration M PO^2 of the white constraint as useless.



(a) A solution of BMM.

(b) The same solution, encoded in the black-white formalism.

Figure 5.6: On the left, a solution of BMM in a 3-regular graph, as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Edges in the matching get the label M, then we need to use P and O to ensure maximality.

5.7 Ruling Sets

Ruling sets are a family of problems parameterized by two positive integers α and β. An (α, β)-ruling set is a set of nodes that satisfies two properties:

- Nodes in the set are at distance at least α from each other.
- Each node not in the set has a node in the set at distance at most β.

Observe that the problem of computing a $(2,1)$-ruling set is the same as MIS. We will now see how to encode $(2,2)$-ruling sets on 3-regular graphs, in the node-edge formalism.

M^3

$\mathsf{P}\ \mathsf{O}^2$

$\mathsf{Q}\ \mathsf{V}^2$

$\mathsf{M}\ \mathsf{PO}$

$\mathsf{O}\ \mathsf{Q}$

OV^2

In other words:

- Nodes in the set output M^3, and M^2 is not in the edge constraint, to satisfy independence.
- Nodes not in the set need to output *pointers* to prove maximality. These pointers can be P or Q. However, P is only compatible with M, and hence nodes outputting $\mathsf{P}\ \mathsf{O}^2$ must be at distance 1 from at least one node in the set. Nodes at distance 2 from nodes in the set output $\mathsf{Q}\ \mathsf{V}^2$, and Q must point to an O, and hence to a node at distance 1, ensuring that all nodes outputting $\mathsf{Q}\ \mathsf{V}^2$ are at distance 2 from nodes in the set.
- Except for needing to output pointers, there are no further constraints, so on the edges we allow any choice over OV^2.

This is not the only way to encode $(2,2)$-ruling sets. We could have for example allowed $\mathsf{M}\ \mathsf{QV}$ on the edges, which would allow nodes outputting $\mathsf{Q}\ \mathsf{V}^2$ to be a distance 1 from nodes in the set. With both encodings, it would take 2 rounds to convert a solution for (standard) $(2,2)$-ruling sets into the version encoded in the node-edge formalism.

If we were to generalize this idea to $(2,\beta)$-ruling sets, for larger β, we would essentially do it as follows:

- We require nodes in the set to output M^3.
- For each $1 \leq i \leq \beta$, we would introduce two new labels P_i and O_i.
- Then, nodes at distance i from the set need to output $\mathsf{P}_i\ \mathsf{O}_i\ \mathsf{O}_i$, and P_i would need to point to O_{i-1} (or to M, if $i = 1$).

However, now observe that, in order to convert a $(2,\beta)$-ruling set into a solution of this encoded version, it would take β rounds of communication, because nodes not in the set need to figure out their distance from the set. We need to be very careful about this when proving lower bounds: suppose we use round elimination to prove an $f(\beta)$ lower bound for the version of the problem encoded in the node-edge formalism, for some function f. This would not give an $f(\beta)$ lower bound for $(2,\beta)$-ruling sets, but only an $f(\beta) - \beta$ lower bound. Depending on f, this could matter a lot.

5.8 Problems vs Family of Problems

Observe that all the considered examples were problems defined on 3-regular graphs. In fact, we used sentences like "let's see how to define, in the node-edge formalism, the problem of computing an MIS on 3-regular graphs". Observe that we never defined "MIS" or "Maximal Matching", we always only defined them on graphs of a fixed degree. In fact, we could define (by considering a suitable generalization of the node-edge formalism) "MIS on graphs of degree ≤ 100", but we *cannot* define MIS in the node-edge formalism. The reason is that there are infinite values of Δ, but we want our problems to have finite description (so that we can apply round elimination on them).

The workaround is the following. We don't consider MIS to be a problem, but we consider it to be a *family of problems*. In other words, we define MIS (on regular graphs) as the family of problems, containing, for each $\Delta \in \mathbb{N}^+$, the following problem:

M^{Δ}
$\mathsf{P}\ \mathsf{O}^{\Delta-1}$

M OP
O O

Yes, we just replaced Δ with a 3. However, think about defining, e.g., Δ-coloring. Then it is not as easy as making some exponents depend on Δ. We would now have that the number of configurations itself depends on Δ, so we would need a new language for describing our problems. This makes it hard to apply the round elimination technique: as we will see, this technique, as is, can be applied on a given problem, but if we want to apply it on an entire family of problems at once, things get more complicated. However, this also shows how we can step outside the LCLs world:

- According to the notation that we just introduced, MIS is definitely *not* an LCL. It is an infinite family of problems, one for each maximum degree Δ.
- We use round elimination to prove, for each problem in the family, a lower bound.
- We see how the asymptotics goes, and we get lower bounds as a function of Δ.

Chapter 6

Generalizations of the Black-White Formalism

[TODO: two ways of define problems with inputs] [TODO: non-regular graphs]

Chapter 7

Round Elimination

Round elimination is a technique that can be used to prove lower (and upper) bounds in the LOCAL model. Informally, round elimination is a function $\mathcal{R}$ that takes as input a problem Π in the black-white formalism and outputs a new problem Π' in the black-white formalism, with the guarantee that Π' is exactly one round easier than Π. This statement is omitting many details that we will see later. However, lower bounds are essentially obtained by applying this function multiple times.

7.1 The function $\mathcal{R}$.

Let $\Pi = (\Sigma, W, B)$. We now see how the problem $\Pi' = (\Sigma', B', W') = \mathcal{R}(\Pi)$ is formally defined. We will later then see some intuition and some examples. First of all, as mentioned earlier, when describing a problem in the black-white formalism, we are implicitly specifying *who* is going to output the labels. That is, while the problems (Σ, W, B) and (Σ, B, W) have exactly the same constraints, in the former case we require white nodes to output labels, and we say that white nodes are active, while black nodes are passive, while in the latter case we require the black nodes to output, and we say that black nodes are active and white nodes are passive. When applying the round elimination function, not only we obtain a problem that is exactly 1 round easier, but we also flip the roles of active and passive. Let's now see the definition of Π'. Let Δ be the degree of the white nodes, and δ be the degree of the black nodes.

- Σ' is going to be a subset of $2^{\Sigma} \setminus \emptyset$.
- We first define B' as a function of B.
  - We first define an auxiliary constraint $\mathfrak{C}$ as the set of all configurations $\{L_1, \ldots, L_\delta\}$ such that, for all $1 \leq i \leq \delta$, it holds that $L_i \subseteq 2^{\Sigma} \setminus \emptyset$, and such that, for all choices $\ell_1 \in L_1, \ldots \ell_\delta \in L_\delta$, the configuration $\{\ell_1, \ldots, \ell_\delta\}$ is in B.
  - We define B' as the maximal configurations in $\mathfrak{C}$, that is, the subset of $\mathfrak{C}$ of configurations $\{L_1, \ldots, L_\delta\}$ such that there is no other configuration $\{L'_1, \ldots, L'_\delta\}$ in $\mathfrak{C}$ such that there is a permutation ϕ of $\{1, \ldots, \delta\}$ such that:
    * For all $1 \leq i \leq \delta$, $L_i \subseteq L'_i$;
    * There is at least one $i \in \{1, \ldots, \delta\}$ such that $L_i \subsetneq L'_i$.
- Σ' is defined as the set of labels that appear at least once in the configurations in $\mathfrak{C}$, that is $\Sigma' = \bigcup_{\mathcal{C} \in \mathfrak{C}} \bigcup_{L \in \mathcal{C}} \{L\}$.
- We now define W' as a function of W and Σ'. We define W' as the set of all configurations $\{L_1, \ldots, L_\delta\}$ such that, for all $1 \leq i \leq \Delta$, it holds that $L_i \in \Sigma'$, and such that there exists a choice $\ell_1 \in L_1, \ldots \ell_\Delta \in L_\Delta$ such that the configuration $\{\ell_1, \ldots, \ell_\Delta\}$ is in W.

7.2 Some terminology

We say that a configuration $L_1, \ldots, L_\delta$ *satisfies the universal quantifier* if for all choices $\ell_1 \in L_1, \ldots \ell_\delta \in L_\delta$, the configuration $\{\ell_1, \ldots, \ell_\delta\}$ is in B. Hence we can say that $\mathfrak{C}$ is the set of all configurations that satisfy the universal quantifier. We say that a configuration is maximal if it is included in B'. A configuration $L_1, \ldots, L_\delta$ is *dominated* by $L'_1, \ldots, L'_\delta$ if there exists a permutation ϕ such that: for all $1 \leq i \leq \delta$, $L_i \subseteq L'_i$, and there is at least one i for which the inclusion is strict. Note that B' is obtained by removing dominated configurations from $\mathfrak{C}$.

Similarly, we say that a configuration $L_1, \ldots, L_\delta$ *satisfies the existential quantifier* if there exists a choice $\ell_1 \in L_1, \ldots \ell_\delta \in L_\delta$ such that the configuration $\{\ell_1, \ldots, \ell_\delta\}$ is in W.

We can now describe Π' more compactly as follows:

- B' is the set of maximal configurations over the alphabet $2^\Sigma \setminus \emptyset$ that satisfy the universal quantifier.
- Let Σ' be the set of labels appearing in B'.
- W' is the set of configuration over Σ' that satisfy the existential quantifier.

7.3 An easy way to compute W'

In order to compute W', there is an easy way that one can follow, which is possible to prove equivalent. Start from W' to be the empty set. Do the following operation in all possible ways. For each configuration in W, replace each label ℓ with a label from Σ' that contains ℓ. Add the resulting configuration to W'.

7.4 Some intuition

Let's unpack the definition and see one piece at a time. It is also useful to see the proof of why Π' is exactly one round easier than Π, to better understand why the definition is the one given. First of all, if we defined $B' = \mathfrak{C}$ (instead of taking only maximal configurations), the obtained problem would still be exactly one round easier. The only reason why we made the definition a bit more complicated is that it often makes the resulting problem "smaller", and hence easier to understand. So let's start by understanding $\mathfrak{C}$. Think of the following:

- We have an algorithm $\mathcal{A}$ that solves Π in T rounds, such that the output is produced by white nodes. We call this algorithm a *white algorithm*.
- Recall that an algorithm in the LOCAL model is just a function that maps the view of a node (that is, its radius-T neighborhood) into an output.
- Hence, a white algorithm does not have to be executed necessarily by a white node. If a black node u gathers the view of a white node v, node u can ask "what would v output?"
- We define a *black algorithm* $\mathcal{A}'$ that takes $T-1$ rounds as follows:
  - Each black node u spends $T-1$ rounds to gather its radius-$(T-1)$ neighborhood $B_{T-1}(u)$.
  - Observe that, if node u gathered $B_{T+1}(u)$ instead of $B_{T-1}(u)$, node u could answer the following question for each white neighbor v: what would v output on the edge $\{u, v\}$? This is doable because $B_{T+1}(u)$ contains $B_T(v)$, and $\mathcal{A}$, when running on v, solely depends on $B_T(v)$.

  - We would like node u to answer such a question, but node u does not have enough information to answer it. However, node u can answer the following question (See Figure 7.1). Consider all possible ways of "extending" $B_{T-1}(u)$ such that $B_T(v)$ becomes fixed. For each such way, what is the output of v? By answering this question, node u would obtain a set of outputs, and we can now try to infer some properties about such outputs. For now, this explains why the labels of Π' are sets of labels of Π.

- Recall that node u answered the question for each neighbor $v_1, \ldots, v_\delta$. So node u now obtained δ sets $L_1, \ldots, L_\delta$, and we can ask, what do these sets satisfy? Could it be that there exists a choice $\ell_1 \in L_1, \ldots, \ell_\delta \in L_\delta$ such that $\{\ell_1, \ldots, \ell_\delta\} \notin B$? We will see that the answer is *no*, and observe that $\mathfrak{C}$ is defined as the constraint containing all configurations *not* satisfying this property. In other words, we say, "we don't know what these sets are going to satisfy, but they definitely do not satisfy this property, so let's take the *easiest* problem not satisfying this property" (recall that $\mathfrak{C}$ contains *allowed* configurations, so by putting *all* configurations *not* satisfying this property, we make the problem as *easy* as possible). Suppose for a contradiction the answer is *yes*. Then, we can take $B_{T-1}(u)$, and δ extensions of it, $Y_1, \ldots, Y_\delta$, and create a graph G satisfying the following: when running $\mathcal{A}$, node v_1 outputs ℓ_1, ..., node v_δ outputs ℓ_δ. We would obtain that $\mathcal{A}$ produces an invalid configuration on u, a contradiction. See Figure 7.2.

- Ignore for a moment that Π' is defined by taking only maximal configurations from $\mathfrak{C}$, and supposed we defined Π' by using $\mathfrak{C}$ directly. We obtained that Π' can be solved in $T-1$ rounds by a black algorithm! This means that Π' is *at least one round easier* than Π.

- Can we also solve the *real* Π' in $T-1$ rounds, the one that does not allow non-maximal configurations? Actually, yes. Suppose node u, by computing $\{L_1, \ldots, L_\delta\}$, obtains a non-maximal configuration. Consider an arbitrary maximal configuration $\{L'_1, \ldots, L'_\delta\}$ satisfying that, for all i, $L_i \subseteq L'_i$. If u outputs such a configuration, it is clearly still satisfying the black constraint. Moreover, since the white nodes need to satisfy an existential quantifier, and since we made each set larger, we obtain that the white constraint is also still satisfied.

- Let's now prove that Π' is *at most one round easier*. We prove this by showing that, given a black algorithm $\mathcal{A}'$ solving Π' in $T-1$ rounds, we can create a white algorithm $\mathcal{A}$ solving Π in T rounds. The algorithm $\mathcal{A}$ simply works as follows.
  - Run $\mathcal{A}'$. This takes $T-1$ rounds.
  - Each black node u sends to each white neighbor v the output that it produced for the edge $\{u, v\}$. This only takes 1 round.
  - Each white node obtained sets $L_1, \ldots, L_\Delta$. By the definition of W', white nodes can pick $\ell_1 \in L_1, \ldots, \ell_\Delta \in L_\Delta$ such that $\ell_1, \ldots, \ell_\Delta$ is in W, and output such a configuration. By the definition of B', since the configurations in B' satisfy the universal quantifier, all black nodes will be happy.

7.5 Some technicalities

Recall that the core idea used for proving that Π' is at least one round easier than Π is the following:

- Assume that, for each i, there is an extension Y_i that, when combined it with $B_{T-1}(u)$, forces v_i to output ℓ_i.

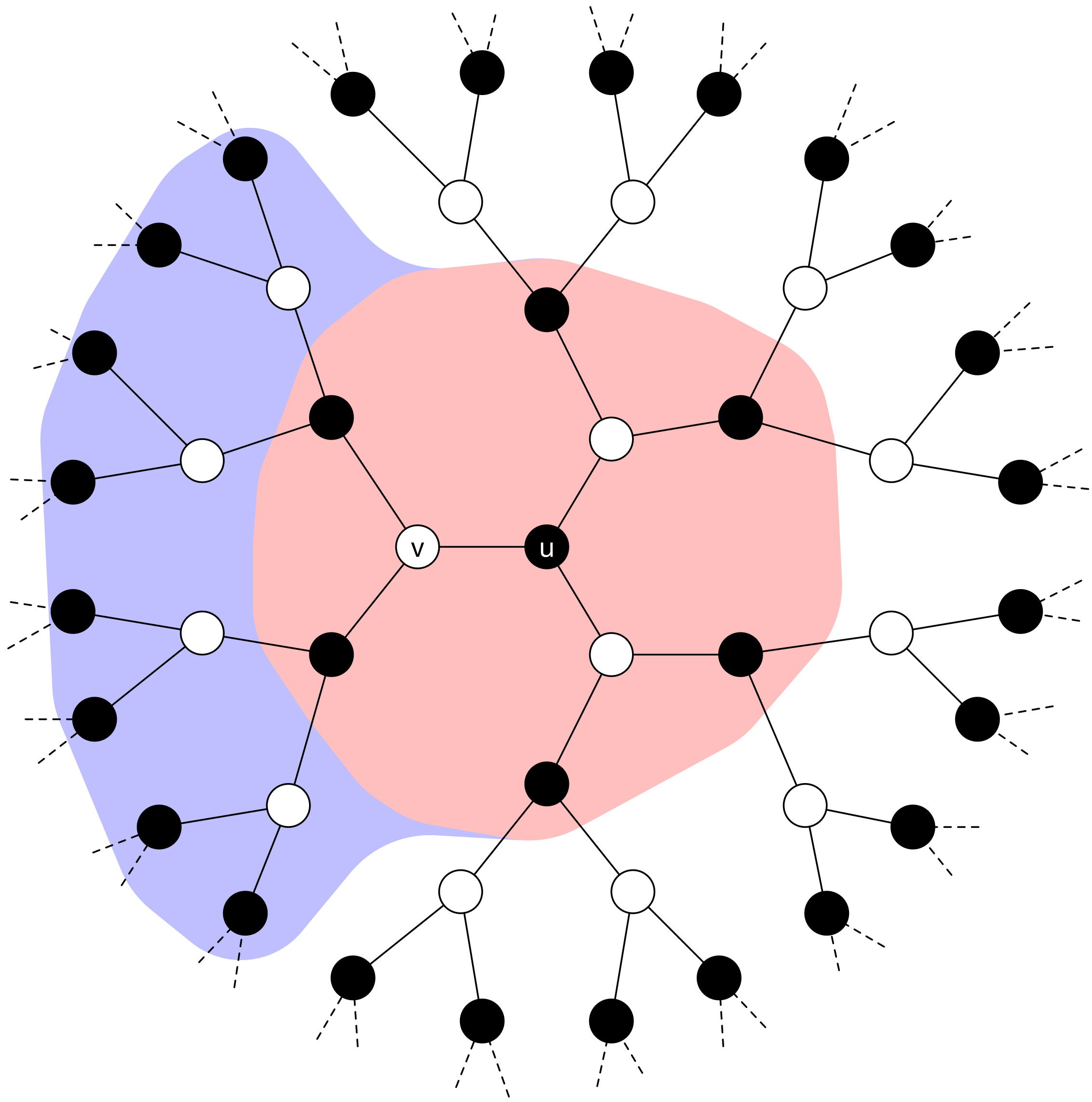


Figure 7.1: Let X be the red area. Let Y be the purple area. Let $T = 3$. An algorithm running on v would only depend on X and Y (note that the union of X and Y is exactly $B_3(v)$). Node u only sees $B_2(u)$, which is exactly X. It does not see Y. Node u, in order to know the output of v, would need to know Y. Node u considers all possible Y (which means that it considers all possible ways in which that part of the graph could look like: all possible inputs, all possible random bit assignments, ...), and for each of them computes the output of node v.

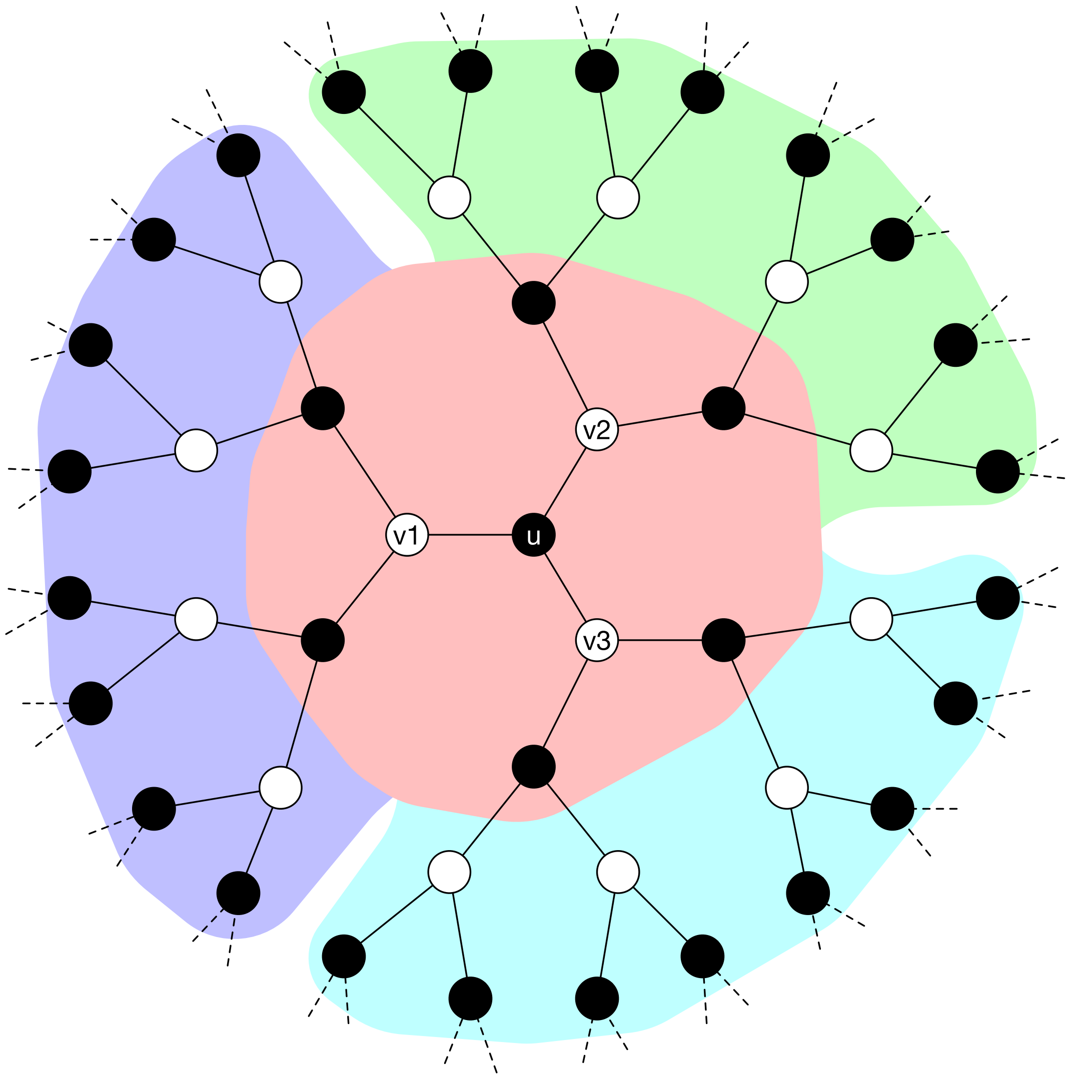


Figure 7.2: Let X be the red area. Let Y_1 be the purple area. Let Y_2 be the green area. Let Y_3 be the cyan area. Let $T = 3$. An algorithm running on v_1 would only depend on X and Y_1 (note that the union of X and Y_1 is exactly $B_3(v_1)$). An algorithm running on v_2 would only depend on X and Y_2 (note that the union of X and Y_2 is exactly $B_3(v_2)$). An algorithm running on v_3 would only depend on X and Y_3 (note that the union of X and Y_3 is exactly $B_3(v_3)$). Suppose there exist Y_1, Y_2, Y_3 such that v_1, v_2, v_3 output a configuration $\{\ell_1, \ell_2, \ell_3\} \notin \mathfrak{C}$. We could create a graph in which $\mathcal{A}$ fails on u. This would contradict the correctness of $\mathcal{A}$.

- Then, we can create a graph G that contains $B_{T-1}(u)$ and all Y_i *at the same time*. Recall Figure 7.2.

However, can we really do that? Imagine that the Y_1 that forces v_1 to output ℓ_1 contains a node with id 123, and that the Y_2 that forces v_2 to output ℓ_2 contains also a node with id 123. Then the graph G that we create would not be a valid instance, because it would contain two distinct nodes with the same id. This is the reason why round elimination does not directly apply to the LOCAL model, and some more machinery is needed. As is, round elimination works in the so-called *deterministic port-numbering model*, which is the same as deterministic LOCAL, but without IDs. This is a weak model, and we would not be satisfied if our lower bounds applied to just this model. There is another issue: suppose $T \gg \log_\Delta n$. For the graph to stay regular, in T rounds of communication, we would necessarily have to see at least one cycle. This could also make Y_1 and Y_1 not independend of each other. This is why round elimination cannot give lower bounds that are asymptotically better than $\log_\Delta n$.

While how to get lower bounds better than $\log_\Delta n$ is still unknown, the other issue is fully solved. The route is the following:

- We first make round elimination work in the *randomized port-numbering model*, which is the same as randomized LOCAL but without IDs. In fact, one can prove that, if there is a white algorithm that solves Π in T rounds with failure probability p, then there is a black algorithm that solves Π in $T-1$ rounds with failure probability p' that is polynomially larger than p. This polynomial evolution in the failure probability is what makes randomized lower bounds obtained via round elimination asymptotically not better than $\log_\Delta \log n$.

- Then, we observe that randomized port-numbering and randomized LOCAL are essentially the same model: in the randomized port-numbering model we can generate unique ids with high probability. Thus, we obtain lower bounds for randomized LOCAL.

- Then, we use some existing techniques to convert a randomized lower bound into a lower bound for deterministic LOCAL that is even better.

- However, we really don't have to worry about all this! There are existing theorems that can be used fully as a black box, to convert a lower bound obtained via round elimination into lower bounds for the LOCAL model. We will discuss this in more detail later.

7.6 An example: edge grabbing

Edge grabbing requires each node to mark *exactly one* incident edge, such that no edge is marked by both endpoints at the same time. Observe that it is essentially the same problem as sinkless orientation:

- Given a solution for sinkless orientation, each edge can mark an arbitrary outgoing edge.

- Given a solution for edge grabbing, nodes can label grabbed edges as *outgoing*, and then they can spend 1 round to know which of the incident edges are grabbed by the neighbors (and label them *incoming*), and to agree on an orientation of the incident edges not grabbed by anybody.

Hence, if we prove a lower bound of T rounds for edge grabbing, we obtain a lower bound of T rounds for sinkless orientation as well. Let's consider this problem on 3-regular graphs. The problem can be defined in the node-edge formalism as follows:

$\mathsf{A}\ \mathsf{B}^2$

$\mathsf{B}\ \mathsf{AB}$

In other words, A means *grabbed*, and B means *not grabbed*. Each node outputs exactly one A. On the edges, at most one side can label the edge A. Recall that the notation above means that $\Pi = (\Sigma, W, B)$ is defined as:

- $\Sigma = \{\mathsf{A}, \mathsf{B}\}$
- $W = \{\{\mathsf{A}, \mathsf{B}, \mathsf{B}\}\}$
- $B = \{\{\mathsf{B}, \mathsf{A}\}, \{\mathsf{B}, \mathsf{B}\}\}$

We will compute $\Pi' = (\Sigma', B', W') = \mathcal{R}(\Pi)$. However, before that, please note the following. Π' would be one round easier than Π in a mode where we add a black node in the middle of each edge. But in the model that we are considering there are no such nodes. There are two (both correct) ways to interpret this:

- We prove a lower bound of T rounds in a model where such black nodes exist. By an easy simulation argument (distances in these two models differ by a multiplicative factor of 2), this implies a lower bound of $T/2$ rounds in a model where such nodes do not exist.
- We are considering a model where edges can be computing entities, and we are saying that if we have a T-round algorithm for Π running on nodes, we obtain an algorithm for Π' in a model where edges are the computing entities and requires $T - \frac{1}{2}$ rounds. By applying $\mathcal{R}$ again, we will go back to a node algorithm that requires $T - 1$ rounds.

It does not matter which interpretation we choose, if we manage to apply $\mathcal{R}$ for x times, in both cases we get a lower bound of $x/2$ rounds for the case in which black nodes do not exist.

Let's compute $\Pi' = (\Sigma', B', W')$.

- We compute B'. Let's first compute the configurations that satisfy the universal quantifier. We obtain the following.

  $\{\mathsf{B}\}\ \{\mathsf{B}\}$
  $\{\mathsf{B}\}\ \{\mathsf{A}\}$
  $\{\mathsf{B}\}\ \{\mathsf{A}, \mathsf{B}\}$

  Indeed, there are exactly all the configurations over the labels $2^{\{\mathsf{A},\mathsf{B}\}} \setminus \emptyset$ that contain A in at most one of the two sets (we need to satisfy this requirement because B allows every configuration except for $\mathsf{A}\ \mathsf{A}$). We now discard dominated (i.e., non-maximal) configurations. The only one remaining is $\{\mathsf{B}\}\ \{\mathsf{A}, \mathsf{B}\}$. Hence, B' is the following constraint.

  $\{\mathsf{B}\}\ \{\mathsf{A}, \mathsf{B}\}$

- We obtain that $\Sigma' = \{\{\mathsf{B}\}, \{\mathsf{A}, \mathsf{B}\}\}$.
- We now compute W', which must contain all configurations from which we can pick one A and two B. Any configuration different from $\{\mathsf{B}\}^3$ would satisfy this requirement. Hence, W' contains the following configurations:

  $\{\mathsf{A}, \mathsf{B}\}\ \{\mathsf{B}\}^2$
  $\{\mathsf{A}, \mathsf{B}\}^2\ \{\mathsf{B}\}$
  $\{\mathsf{A}, \mathsf{B}\}^3$

We computed Π'. However, let us make it easier to read. Let's rename some labels. Let use replace $\{\mathsf{B}\}$ with A and $\{\mathsf{A}, \mathsf{B}\}$ with B. We obtain that Π' is the following problem:

$\mathsf{A}\ \mathsf{B}$

$\mathsf{B}\ \mathsf{A}^2$

$\mathsf{B}^2\ \mathsf{A}$

B^3

We can write this problem more compactly as:

$\mathsf{A}\ \mathsf{B}$

$\mathsf{B}\ \mathsf{AB}^2$

Let's now compute $\Pi'' = (\Sigma'', W'', B'') = \mathcal{R}(\Pi')$.

- We first compute W''. By the definition of W', we want all configurations over $2^{\Sigma'} \setminus \emptyset$ from which we can pick at least one B. If we consider only the maximal ones, we get that the only configuration contained in W'' is the following:

  $\{\mathsf{B}\}\ \{\mathsf{A}, \mathsf{B}\}^2$

- Let's now compute B'' by using the easy method. We start from $\mathsf{A}\ \mathsf{B}$ and we replace A with all sets containing A (which is only $\{\mathsf{A}, \mathsf{B}\}$) and we replace B with all sets containing B (which are $\{\mathsf{B}\}$ and $\{\mathsf{A}, \mathsf{B}\}$). We obtain the following condensed configuration:

  $\{\mathsf{A}, \mathsf{B}\}\ \{\mathsf{B}\}\{\mathsf{A}, \mathsf{B}\}$

As before, let us rename the labels to make the problem easier to read. Let's rename some labels. Let use replace $\{\mathsf{B}\}$ with A and $\{\mathsf{A}, \mathsf{B}\}$ with B. We obtain that Π'' is the following problem:

$\mathsf{A}\ \mathsf{B}^2$

$\mathsf{B}\ \mathsf{AB}$

We obtained that $\Pi'' = \Pi$. There must be something wrong. How can Π be exactly 2 rounds easier than itself? There is nothing wrong: remember the small technicalities mentioned in Section 7.5. We can interpret what we obtained as follows:

> Suppose Π can be solved in T rounds, such that T is small-enough so that the requirements of round elimination apply (that is, there are no cycles in the radius-T neighborhood of the nodes). Then, Π is 2 rounds easier than itself. Since the conclusion is a contradiction, the premise is false, hence Π cannot be solved with small-enough T.

As said earlier, there are some black-box theorems that we can apply. One of them is the following:

Theorem 1 *Suppose that* $\Pi = \mathcal{R}(\Pi)$ *or* $\Pi = \mathcal{R}(\mathcal{R}(\Pi))$. *Then,* Π *requires* $\Omega(\log n)$ *for deterministic algorithms in the LOCAL model, and* $\Omega(\log \log n)$ *for randomized ones.*

We thus get that edge grabbing, and sinkless orientation, in 3-regular graphs, have deterministic and randomized lower bounds of $\Omega(\log n)$ and $\Omega(\log \log n)$ respectively. As a side note, such lower bounds (and all lower bounds obtained via round elimination) also apply to regular balanced trees as well. Another note: the Ω notation in Theorem 1 is hiding dependencies in Δ and δ. While for our specific case it does not matter (because we are considering the problem on 3-regular graphs), suppose we tried to prove a lower bound for edge grabbing in Δ-regular graphs as a function of n and Δ. In other words, suppose we tried to prove a lower bound for a whole family of problems at the same time. Then we would have needed to use a different lifting theorem, that correctly takes Δ into account.

7.7 Fixed Points

What we have seen in Section 7.6 is an example of so-called *fixed point*, and in particular a *non-trivial* fixed point. Non-trivial here just means that the fixed point is not solvable in 0 rounds of communication. I.e., the problem

A^3

A^2

is also a fixed point, but it is a trivial problem: everybody can output A on all its incident edges.

As previously discussed, LCL problems can be classified as being *easy* or *hard*. In order to prove that a problem is easy, it is sufficient to provide an $O(\log^\star n)$ algorithm. In order to prove that a problem is hard, we need to provide a deterministic $\Omega(\log n)$ lower bound. As discussed, showing that a problem is a non-trivial fixed point implies that the problem is $\Omega(\log n)$. However, very few interesting problem are non-trivial fixed points. A technique that we often use to prove an $\Omega(\log n)$ lower bound is to provide a non-trivial fixed point *relaxation*, that is, given a problem Π for which we want to prove a lower bound, we find a problem Π' that is *at least as easy as* Π and such that Π' is a non-trivial fixed point. Providing a fixed point relaxation for a problem is one of the very few know ways that we know to prove that the problem is $\Omega(\log n)$. A major open question is the following:

Open Problem 1 *Does every* $\Omega(\log n)$ *problem admit a non-trivial fixed point relaxation?*

In other words, we don't know if, providing a non-trivial fixed point relaxation for a problem, is a universal way to prove lower bounds. Any answer to Open Problem 1 would have important consequences:

- If the answer is "yes", it would imply *decidability*. In more detail, it is known that it is semi-decidable whether a problem is solvable in $O(\log^\star n)$ rounds. Since it is semi-decidable whether a problem admits a non-trivial fixed point relaxation, if the answer is "yes" then we would get that it is semi-decidable whether the problem is $\Omega(\log n)$. Combining the two, we would get that we can decide if a problem is easy or hard.

- If the answer is "no", then it means that there are problems that are $\Omega(\log n)$ for which we have no known ways to prove that they are indeed $\Omega(\log n)$.

Currently, we have *zero* problems that we know to be $\Omega(\log n)$ but for which we do not know a non-trivial fixed point relaxation.

In the past, we had some. There is a technique that, in the distributed setting, is called Mark's technique, and comes from a different field called *descriptive combinatorics*. This technique can be used to prove that a problem does not belong to a class of problems called *Borel*. It is known that if a problem is not in this class, then the problem requires $\Omega(\log n)$ (note, it is not an if and only if). For quite a while we knew of a class of problems called *homomorphism problems*, for which Mark's technique succeeds in giving lower bounds. These problems were candidates for answering "no" to Open Problem 1. We now know that, if Mark's technique succeeds for a problem, then we can always construct a non-trivial fixed point relaxation for the problem. So currently we have zero candidates that could help in answering "no" to the question, and we know that such candidates cannot be obtained via Mark's technique. However, we know that the answer *is* "no" for LCLs with inputs. In more detail, consider the problem of computing a sinkless and sourceless orientation (SSO) given as input a solution for sinkless orientation (SO), that we will call SSO-given-SO. We know a non-trivial fixed point relaxation for SSO (which is edge grabbing, EG). However, EG is a trivial problem in the context in which a solution for SO is given as input. It has been proved that *any* non-trivial fixed point relaxation of SSO *is trivial* given a solution for SO. So how do we know that SSO-given-SO is $\Omega(\log n)$? Well, providing a fixed point is not really the only way to prove such a lower bound:

- SSO behaves very nicely under round elimination, the number of labels grows by 1 at each $\mathcal{R}$ step.
- There are lifting theorems that say that if we have an infinite sequence of problems given by $\mathcal{R}$ that are all non-trivial, where the number of labels grows "slow-enough", then the initial problem is $\Omega(\log n)$.

But SSO is really an exception, the typical behavior of an interesting problem Π is the following:

- If we try to apply round elimination to Π, we see that, at each step, the number of labels grows exponentially.
- We try to *relax* Π, or $\mathcal{R}(\Pi)$, or $\mathcal{R}(\mathcal{R}(\Pi))$, ..., with the hope to obtain a non-trivial fixed point.
- In all known cases, if we reach a problem that behaves as nicely as SSO (i.e., linear growth in the number of labels), then we can relax the problem a bit more and get a non-trivial fixed point.

So the only way this idea of "we can't get a fixed point but linear growth is enough for a lower bound" has been used is to prove a lower bound in the setting with input. In fact, a related open question is the following:

Open Problem 2 *Is there a universal technique that can be used to prove lower bound in the setting of LCLs with input?*

We know that "fixed points" are not the answer, because of SSO-given-SO. And it seems unlikely that "linear growth" is the answer. An example of problem for which we know how to get lower bounds via reductions but not directly via round elimination is, e.g., the problem Π of computing a 4-coloring in 4-regular graphs, given an orientation where every node has exactly 2 incoming and 2 outgoing edges. For this problem, we do not know whether it is possible to define a sequence of problems $\Pi_1, \ldots,$ such that $\Pi_1 = \Pi$ and Π_{i+1} is a relaxation of $\mathcal{R}(\Pi_i)$ and such that the growth in the number of labels is not exponential.

For now, we will put the topic of inputs aside, and we will discuss fixed points in the setting with no input. We will see some examples of fixed points and of non-trivial fixed point relaxations.

7.7.1 Example: 2-coloring

Consider the problem of 2-color a 3-regular graph. In the node-edge formalism, it can be defined as follows:

A^3

B^3

$\mathsf{A}\ \mathsf{B}$

An easy way to see that it is a fixed point is the following: if we pick any pair of configurations from the node side (including different permutations of the same configuration), or any pair on the edge side, the edit distance between them is ≥ 2. This implies that, when we write down the configurations satisfying the universal quantifier, we only get the original configurations, where we replace labels with singleton sets. This ensures that the problems does not change when applying round elimination, and after two steps of $\mathcal{R}$ we get back the problem itself.

7.7.2 Example: perfect matching

Consider the problem of computing a perfect matching in a 3-regular graph. In the node-edge formalism, it can be defined as follows:

$\mathsf{M}\ \mathsf{O}^2$

$\mathsf{M}\ \mathsf{M}$

$\mathsf{O}\ \mathsf{O}$

In other words, each node marks exactly one edge, and an edge must be either marked by both endpoints or by none of them. For the same reason as 2-coloring, this problem is a fixed point.

Problems that satisfy the aforementioned edit-distance ≥ 2 property are typically only problems that are so hard that one could prove lower bounds via different techniques. The more typical case is that the problem of interest is not a fixed point, and we need to find a non-trivial fixed point relaxation for it.

7.7.3 Example: sinkless orientation

Sinkless orientation on 3-regular graphs can be defined in the node-edge formalism as follows:

$\mathsf{A}\ \mathsf{AB}^2$

$\mathsf{A}\ \mathsf{B}$

Note that this problem does not satisfy the edit-distance ≥ 2 property: on the nodes, we could pick A^3 and $\mathsf{A}^2\ \mathsf{B}$, which have edit distance 1. One can also check that this problem is not a fixed point. In fact, one can check that $\mathcal{R}(\mathcal{R}(\text{SO})) = \text{EG}$, and note that $\text{EG} \neq \text{SO}$. But we have already seen that EG is a fixed point in Section 7.6. Since $\mathcal{R}(\mathcal{R}(\text{SO}))$ is a relaxation of SO (the $\mathcal{R}$ operator guarantees that), we get that EG is a fixed point relaxation of SO. One can also check that EG is a non-trivial problem (we will later see how), and we hence get that EG is a non-trivial fixed point relaxation of SO. Unfortunately, this is really an exceptional case, problems do not tend to naturally become fixed points after few steps of $\mathcal{R}$.

7.7.4 Example: sinkless and sourceless orientation

Sinkless and sourceless orientation on 3-regular graphs can be defined in the node-edge formalism as follows:

$\mathsf{A}\ \mathsf{B}\ \mathsf{AB}$

$\mathsf{A}\ \mathsf{B}$

That is, each edge must be properly oriented, and each node needs to have at least one incoming and at least one outgoing edge. This problem is not a fixed point, and one can check that this problem would not converge into a fixed point by applying $\mathcal{R}$ (the number of labels would grow by 1 at each step). However, there is a really easy relaxation that we can perform: we add A^3 to the allowed node configurations. We get that the node constraint is

$\mathsf{A}\ \mathsf{B}\ \mathsf{AB}$

A^3

which can be also written as

$\mathsf{A}\ \mathsf{AB}^2$

which is the same node constraint as sinkless orientation. In other words, we can relax SSO to SO, and SO to EG, and EG is a non-trivial fixed point. We get that SSO admits a non-trivial fixed point relaxation.

7.7.5 Example: 3-coloring

Consider the problem of 3-coloring 3-regular graphs. It can be defined as follows:

A^3

B^3

C^3

$\mathsf{A}\ \mathsf{BC}$

$\mathsf{B}\ \mathsf{C}$

This is an example of a problem for which, finding a non-trivial fixed point relaxation, is not entirely trivial. In fact, we knew that this problem was hard even before knowing a fixed point for it, and it was proved as follows:

- Consider the setting in which the graph comes with a 3-edge coloring given.
- One can prove that, in such a setting, EG is still non-trivial. Not only, one can also prove that, in such a setting, EG is still *hard*.
- However, given a 3-coloring *and* a 3-edge coloring, EG becomes 0-round solvable (nodes of color x grab the edge with color x).
- Moreover, if we are only given a 3-coloring, *but not* a 3-edge coloring, then EG is still non-trivial.
- This suffices to infer that 3-coloring is a hard problem, that is, it requires $\Omega(\log n)$ deterministic rounds. In fact, suppose for a contradiction that 3-coloring was easy. We would get that, given a 3-edge coloring, we could solve an easy problem (3-coloring) and then use the obtained solution to solve EG, but we know that EG is hard given a 3-edge coloring.

A non-trivial fixed point relaxation for 3-coloring came much later. A possible reason is the following: if we take the natural generalization of this problem, that is, Δ-coloring Δ-regular graphs, the "smallest" non-trivial fixed point relaxation that we know is a problem that requires 2^Δ labels to be described. So, in some sense, to get a fixed point we needed to guess a problem that has exponentially larger description size.

A non-trivial fixed point relaxation for 3-coloring is the following. We use some new notation here. (AB) represents a single label. That is, if we want to give a name to a label and use multiple characters, we put those character between parentheses.

A^3
B^3
C^3
$(\mathsf{AB})^2\ \mathsf{E}$
$(\mathsf{AC})^2\ \mathsf{E}$
$(\mathsf{BC})^2\ \mathsf{E}$
$(\mathsf{ABC})\ \mathsf{E}^2$

E EABC(AB)(AC)(BC)(ABC)
A EBC(BC)
B EAC(AC)
C EAB(AB)
(AB) EC
(AC) EB
(BC) EA
(ABC) E

Note that some configurations have been written multiple times, just to more easily see the pattern. Think of the label E as "empty", or as "no color", and more generally think of each label as a set of colors. For example A is the set containing only color A, while (AB) is the set containing colors A and B, while E is the empty set. We can see that the problem allows the following solutions:

- Nodes can output a color, as in the case of 3-coloring, or
- Nodes can output multiple colors. However, if they output k colors, then they can output the empty set on $k-1$ incident half-edges. For example, if a node outputs $\mathsf{AB}^2\ \mathsf{E}$, then we can interptet it as being colored A and B at the same time.
- The edge constraint requires the endpoints of an edge to have assigned disjoint sets.

In other words, this is a generalization of coloring. In standard coloring, each node outputs exactly one color. In this version, nodes may output multiple colors. Note that outputting multiple colors is a hard task: if a node outputs (AB) on an edge, the other endpoint cannot output neither A nor B on the same edge. However, nodes that output multiple colors are rewarded: for each additional color that they output (except for the first), they can mark an incident edge as "don't care". One can prove that this problem is a fixed point and is not trivial.

Summarizing, 3-coloring is not a fixed point, but we can find a non-trivial fixed point relaxation of it that requires 8 labels to be described, and this fixed point has a natural interpretation.

7.7.6 Example: locally optimal cut

Not all fixed points admit a natural interpretation. We will now discuss a problem for which proving an $\Omega(\log n)$ lower bound has been significantly harder. The problem is the one of computing a locally optimal cut (LOC) in 3-regular graphs. A locally optimal cut is a 2-coloring of the nodes satisfying that, if a node has a color, it needs to have at least two neighbors of the opposite color. It is a locally optimal cut in a game-theoretic sense: if nodes try to selfishly maximize the number of neighbors of the opposite color by unilaterally deciding to change color, then this problem is a Nash-stable solution. This problem is also called 1-defective 2-coloring: it is a 2-coloring where nodes are allowed to have at most 1 neighbor of the same color. This problem is also called unfriendly partition or majority coloring. In the node-edge formalism, this problem can be encoded as follows.

$\mathsf{A}^2\ \mathsf{X}$
$\mathsf{B}^2\ \mathsf{Y}$

$\mathsf{AX}\ \mathsf{BY}$
$\mathsf{XY}\ \mathsf{XY}$

That is, nodes of one color output $\mathsf{A}^2\ \mathsf{X}$, while nodes of the other color output $\mathsf{B}^2\ Y$. The labels X and Y are used to mark the edge in which there could be a neighbor of the opposite color. In fact, label X is compatible with X (so two neighbors of the first color can both label their connecting edge with X) but not with A (as otherwise a node of the first color could end up having more than 1 neighbor of the first color), and is also compatible with both B and Y (because they are both labels used by nodes of the opposite color).

The original proof showing that this problem is $\Omega(\log n)$ uses a reduction from the hardness of sinkless orientation. This reduction is non-trivial. In fact, we cannot directly convert a solution for LOC into a solution for SO, and the proof follows a different route. The proof explots the fact that, if a problem can be solved in $o(\log n)$, then it can also be solved in $O(\log^\star n)$ in a normal form. The normal form proceeds as follows:

- For some suitable parameters c and d, compute a distance-d c-coloring, that is, a coloring with c colors such that nodes within distance d have different colors.
- Apply an $O(1)$-round algorithm after that.

The lower bound is obtained by showing that, if we have an $o(\log n)$ rounds algorithm for LOC, and we normalize it in this way, then we can construct a suitable virtual graph where we run this algorithm, and we can convert the obtained solution into a sinkless orientation solution on the original graph.

For quite a white we did not have an alternative proof of hardness, and this problem was a candidate to answer “no” to Open Problem 1. However, we eventually got a fixed point for this problem, which is the following:

$\mathsf{(AX)}^2\ \mathsf{X}$
$\mathsf{(BY)}^2\ \mathsf{Y}$
$\mathsf{(ABXY+)}\ \mathsf{(XY)}\ \emptyset$
$\mathsf{(BXY+)}\ \mathsf{(AXY+)}\ \emptyset$
$\mathsf{(AXY+)}\ \mathsf{X}^2$
$\mathsf{(BXY+)}\ \mathsf{Y}^2$
$\mathsf{(XY+)}\ \mathsf{(XY)}^2$

$\mathsf{X(AX)}\emptyset\ \mathsf{(BY)}\emptyset\mathsf{Y}$
$\emptyset\ \mathsf{X(AX)(BY)(XY)(XY+)}\emptyset\mathsf{(BXY+)(ABXY+)(AXY+)Y}$
$\mathsf{X}\emptyset\ \mathsf{X(BY)(XY)(XY+)}\emptyset\mathsf{(BXY+)Y}$
$\mathsf{X(AX)(XY)(XY+)}\emptyset\mathsf{(AXY+)Y}\ \emptyset\mathsf{Y}$
$\mathsf{X(XY)}\emptyset\mathsf{Y}\ \mathsf{X(XY)(XY+)}\emptyset\mathsf{Y}$

This is a relaxation of the original problem: if we rename $\mathsf{(AX)}$ to A and $\mathsf{(BY)}$ to B, we get that the allowed configurations of LOC are allowed also here. However, differently from the case of the fixed point relaxation of 3-coloring, it is quite hard to get some intuition about this problem. Does it have some natural interpretation? We don't know. This fixed point was found by hand, by manually trying to relax the problems obtained by applying $\mathcal{R}$ on LOC. However, by carefully analyzing this fixed point, one can actually find some very hidden pattern. In fact, there is a mechanical way that one can use to obtained this fixed point by starting from LOC. We will later discuss this mechanical procedure. The same procedure, applied to 3-coloring, gives the fixed point relaxation of 3-coloring that we have seen earlier. In fact, all known fixed points can be obtained by applying this procedure, and discovering this procedure allowed us to find non-trivial fixed point relaxations that are so complicated (hundreds of labels) that finding them manually would have been hopeless.

7.8 Relaxations

We now put the discussion about fixed points aside, and we go back to the basics. We are given a problem Π, how can we try to construct a problem sequence that gives a lower bound? If we compute $\mathcal{R}(\Pi)$, and then $\mathcal{R}(\mathcal{R}(\Pi))$ and so on, for most of the problems of interest we quickly get so many labels that it is hard to find a natural interpretation of the problem, or to even compute the next problem. The basic goal to apply round elimination successfully is to manage to do the following. We want to replace $\mathcal{R}(\Pi)$ with some problem Π' such that:

1. Π' must be at least as easy as Π.
2. Π' has much smaller description size (number of labels, number of configurations, ...) compared to $\mathcal{R}(\Pi)$.
3. Π' must not be *too easy* compared to $\mathcal{R}(\Pi)$.

We want to do this multiple times, and construct a sequence $(\Pi_i)_{i\geq 1}$ satisfying the following:

- $\Pi = \Pi_1$
- Π_{i+1} is at least as easy as $\mathcal{R}(\Pi_i)$

- Let k be the largest k such that Π_k is non-trivial (not 0-rounds solvable). We obtain that Π requires at least k rounds.

Observe that requirement 1 is needed so that the non-triviality of Π_k indeed implies a lower bound for Π. Requirement 3 is needed because, if for some i Π_{i+1} is much easier than $\mathcal{R}(\Pi_i)$, then we cannot hope to get a tight lower bound. In fact, imagine the real complexity of Π to be T, and suppose that Π_2 is $T-5$ rounds easier than $\mathcal{R}(\Pi)$. Then we cannot hope to get a lower bound which is better than 5. But maybe T was much larger than 5. Requirement 2 is needed to keep the whole operation tractable. In fact, we typically have a stronger goal than just keep the problems small. The *real* goal is to be able to describe the whole sequence $\Pi_1, \ldots$ in a closed form. An example of a convenient closed form is a problem where some exponents depend on the index i of the problem. We will see some examples soon. Before that, we first need to discuss how we can actually make a problem *easier and smaller* at the same time. In fact, there is a simple way to make a problem smaller: just discard some labels, and all configurations containing those labels. Unfortunately, this would make a problem *harder*, not easier, because we are removing allowed configurations. However, there is something similar that we can do, that makes a problem *easier*: merging labels. Let us see an example with LOC, and in particular we see what it means to merge X and Y. We modify the problem so that, every time X is allowed, we allow also label Y, and vice versa. We obtain the following problem Π'.

$\mathsf{A}^2\ \mathsf{XY}$
$\mathsf{B}^2\ \mathsf{XY}$

AXY BXY
XY XY

In a solution for this problem, we can always replace X with Y and vice versa (e.g., if a node uses the configuration $\mathsf{A}^2\ \mathsf{X}$, and we change it to $\mathsf{A}^2\ \mathsf{Y}$, we still obtain a valid solution). Hence, the following problem Π'' has the exact same complexity:

$\mathsf{A}^2\ \mathsf{X}$
$\mathsf{B}^2\ \mathsf{X}$

AX BX

In fact, a solution for Π'' is also a solution for Π', and a solution for Π' can be converted in 0 rounds of communication into a solution for Π'', by just replacing every Y with X. The obtained problem Π'' is called 1-arbdefective 2-coloring, and it turns out to be solvable in just 1 round. Suppose we were trying to prove a lower bound for LOC, and suppose that, in order to keep things simpler, we replaced LOC with Π'', which is a relaxation of LOC. Since Π'' is solvable in 1 round, it would have not been possible to obtain a non-trivial fixed point starting from Π''.

> In a very simplified way, the act of finding lower bounds via round elimination boils down to find the right relaxations so that we obtain a sequence of problems that are very similar to each other, so that we can describe all of them as a single problem parameterized by some parameter i.

Is there a way to detect if a relaxation is good, or do we have to try all possible relaxations by brute force? We will discuss this later. We will now discuss some useful ingredients.

7.8.1 Zero-round Solvability

7.8.2 Diagram

[TODO: 0 round solvability]

Chapter 8

Other Lower Bound Techniques

[TODO: Marks, KMW, indistinguishability, ...]

Bibliography

[1] Alkida Balliu, Sebastian Brandt, Yi-Jun Chang, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. The distributed complexity of locally checkable problems on paths is decidable. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 262–271. ACM, 2019.

[2] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. Almost global problems in the LOCAL model. In Ulrich Schmid and Josef Widder, editors, *32nd International Symposium on Distributed Computing, DISC 2018, New Orleans, LA, USA, October 15-19, 2018*, volume 121 of *LIPIcs*, pages 9:1–9:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[3] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. How much does randomness help with locally checkable problems? In Yuval Emek and Christian Cachin, editors, *PODC '20: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3-7, 2020*, pages 299–308. ACM, 2020.

[4] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally checkable labelings with small messages. In Seth Gilbert, editor, *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference)*, volume 209 of *LIPIcs*, pages 8:1–8:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[5] Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. New classes of distributed time complexity. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2018, Los Angeles, CA, USA, June 25-29, 2018*, pages 1307–1318. ACM, 2018.

[6] Sebastian Brandt. An automatic speedup theorem for distributed problems. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 379–388. ACM, 2019.

[7] Sebastian Brandt, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Patric R. J. Östergård, Christopher Purcell, Joel Rybicki, Jukka Suomela, and Przemyslaw Uznanski. LCL problems on grids. In Elad Michael Schiller and Alexander A. Schwarzmann, editors, *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*, pages 101–110. ACM, 2017.

[8] Yi-Jun Chang. The complexity landscape of distributed locally checkable problems on trees. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, volume 179 of *LIPIcs*, pages 18:1–18:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[9] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. An exponential separation between randomized and deterministic complexity in the LOCAL model. In Irit Dinur, editor, *IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA*, pages 615–624. IEEE Computer Society, 2016.

[10] Yi-Jun Chang and Seth Pettie. A time hierarchy theorem for the LOCAL model. *SIAM Journal on Computing*, 48(1):33–69, 2019.

[11] Yi-Jun Chang, Jan Studený, and Jukka Suomela. Distributed graph problems through an automata-theoretic lens. In Tomasz Jurdzinski and Stefan Schmid, editors, *Structural Information and Communication Complexity - 28th International Colloquium, SIROCCO 2021, Wrocław, Poland, June 28 - July 1, 2021, Proceedings*, volume 12810 of *Lecture Notes in Computer Science*, pages 31–49. Springer, 2021.

[12] Moni Naor and Larry J. Stockmeyer. What can be computed locally? *SIAM Journal on Computing*, 24(6):1259–1277, 1995.